

16 экспертов были уверены, что AI их ускоряет — и ошиблись на знак.



Эксперимент METR · RCT · первая половина 2025

16 опытных open-source разработчиков · 246 реальных задач в своих знакомых репозиториях · измеряли реальное время, не ощущение

Три числа об одном и том же

Прогноз до эксперимента

ждали ускорение

-24%

Вера после, уже поработав

думали, что ускорил

≈ -20%

Измеренный факт

по времени — замедление

+19%

? perception-gap

разрыв между субъективным ощущением скорости («AI меня ускоряет») и объективно измеренным фактом

*Ускоряет ли AI лично вас — на сколько?
И откуда вы это знаете?*

Профессионалы, годами пишущие код, ошиблись не в величине — в знаке. «Мне кажется, инструмент помогает» — это гипотеза, а не данные.

ЛЕКЦИЯ 4

AI в разработке программного обеспечения

04

Где AI ускоряет, где вредит —
и что в работе инженера НЕ делегируется

Раздел 0
Открытие

Раздел 1
Уровни A+B

Раздел 2
Уровень C

Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Лекция 4 не вводит новый аппарат — она применяет аппарат Лекции 3.

Переносим готовым, не переобъясняем: 4 блока обзорного Модуля 1 → их проекция на разработку ПО.

Из обзорного Модуля 1 (Лекции 1–3)	→ Проекция на разработку ПО (эта лекция)
Лестница сложности: оставайся на нижней ступени, поднимайся под требование	Лестница автономности $A \rightarrow D$ — та же лестница, та же логика подъёма
«Когда не ИИ вовсе»: детерминированное → обычный код	Критерий «не AI» применяем в каждом разделе
Цикл агента: планируй → делай → проверяй → повторяй	Кодинг-агент (уровень C) = ровно этот цикл, применённый к коду
Prompt injection (внедрение команды через недоверенный контент)	CamoLeak — та же атака, но внутри dev-агента

$A \rightarrow D$ — это лестница сложности Лекции 3, применённая к одной индустрии. Правило не меняется: чем выше уровень автономии, тем строже критерий «здесь обязателен человек».

Центральный вопрос лекции.



«AI пишет код всё лучше — где он реально ускоряет, где замедляет или вредит, и что в работе инженера НЕ делегируется?»

К второй половине вопроса — «где человек обязателен» — возвращаемся 5 раз:

1

«почти
правильный» код

2

merge
без чтения

3

деструктив +
перекос ощущения

4

уязвимость +
утечка

5

что строить
(Brooks / DORA)

Ответ — не «AI хорош» и не «AI плох», а аппарат: для конкретной задачи назвать уровень автономии, конфигурацию и точку обязательного человеческого контроля. Главное в работе инженера — то, что НЕ делегируется.

Одна рамка для всех четырёх уровней.

Снимает главный риск перечисления уровней — превращение их в список инструментов вместо аппарата суждения.



1. Что делает AI

какой объём работы AI выполняет самостоятельно на этом уровне



2. Кто решает

кто принимает содержательное решение — что строить, что принять, что слить



3. Где человек обязателен

точка, где отсутствие человека превращает уровень из инструмента в источник катастрофы



4. Типичный риск

характерный режим отказа именно этого уровня

Не повышай уровень автономии AI без явного требования задачи, которого уровень ниже не закрывает — каждый подъём оплачивается ослаблением человеческого контроля над тем, что попадает в кодовую базу. Это отраслевая проекция правила лестницы Лекции 3.

Самый безопасный уровень безопасен только пока человек реально читает.

Автодополнение — AI дописывает строку/блок по контексту файла; человек принимает или отклоняет каждое предложение в момент написания.

Один инструмент — три разных эффекта на скорость

Лаборатория — изолированная новая задача

+56%

Поле — Microsoft / Accenture

+7...22%

Знакомое легаси — эксперты (METR)

-19%

Рамка уровня А

Что делает AI:	предлагает продолжение по контексту
Кто решает:	человек, на каждом предложении
Где обязателен:	езде — максимум контроля
Типичный риск:	автопринятие без чтения

Сними чтение — и самый безопасный уровень становится поставщиком техдолга на скорости набора текста. Gold-факт: -19% на знакомом легаси у экспертов (METR).

Граница А ↔ В — первое реальное делегирование: человек проверяет после, а не во время.

	Уровень А	Уровень В
Единица работы	строка-в-потоке	задача-фрагмент (функция, фикс)
Где человек в цикле	на каждом токене	после генерации
Когда ревью	в момент написания	отдельным шагом

Смещение «человек проверяет после, а не во время» — первое реальное делегирование в лестнице. С него начинаются характерные проблемы AI-кода.

Рамка уровня В			
Что делает AI:	генерирует законченный фрагмент по задаче	Кто решает:	человек ставит задачу и принимает результат
Где обязателен:	на ревью результата до интеграции	Типичный риск:	«почти правильный» код — разберём далее

Последние 20–30% так же трудны, как были до AI.

70/80%-проблема

- Первые 70–80% — быстро и дёшево: типовое, было в обучающих данных
- Последние 20–30% — краевые случаи, ошибки, безопасность, интеграция, нагрузка — так же трудны
- Разрыв структурный, не временный: специфики системы в обучающих данных не было и быть не могло

66%

разработчиков (опрос Stack Overflow, 2025)

топ-фрустрация — код «почти правильный, но не совсем». Дороже явно неверного: компилируется, проходит обычный сценарий — ломается в проде.

Урок: «AI написал за минуту» без «и я проверил» = «долг записан за минуту». Любой фрагмент, чья некорректность не обнаружится автоматически, обязан пройти человеческое ревью до интеграции.

Не запрет, а инженерный паттерн: structure + constraints + tests.

Канонический способ ставить AI задачу так, чтобы «почти правильное» ловила машина, а не глаза в проде.

Structure

дать сигнатуры, типы, контракт интерфейса, входы/выходы — а не «сделай мне X». Чем строже структура, тем уже пространство правдоподобно-неверного.

Constraints

явно перечислить, чего нельзя и какие инварианты держать. Модель не выведет их сама — она не знает контекста системы.

Tests

тест как исполняемая спецификация — машинно-проверяемый критерий «правильно/нет», не подвержен ни «почти правильному», ни perception-gap.

Это, по сути, TDD (разработка через тесты), применённый к постановке задачи для AI. Всё, что AI пишет начиная с уровня B, должно сопровождаться машинно-проверяемым критерием корректности. Антипаттерн — vibe-coding: генерировать и принимать код «по ощущению», без структуры, ограничений, теста и проверочного шага (строгое определение — дальше в лекции).

РАЗДЕЛ 2

Уровень С: кодинг-агент

На А и В человек видел каждый фрагмент. Уровень С — AI сам планирует, правит много файлов, гоняет тесты; человек видит только итог — pull request.



Раздел 0
Открытие

Раздел 1
Уровни А+В

Раздел 2
Уровень С

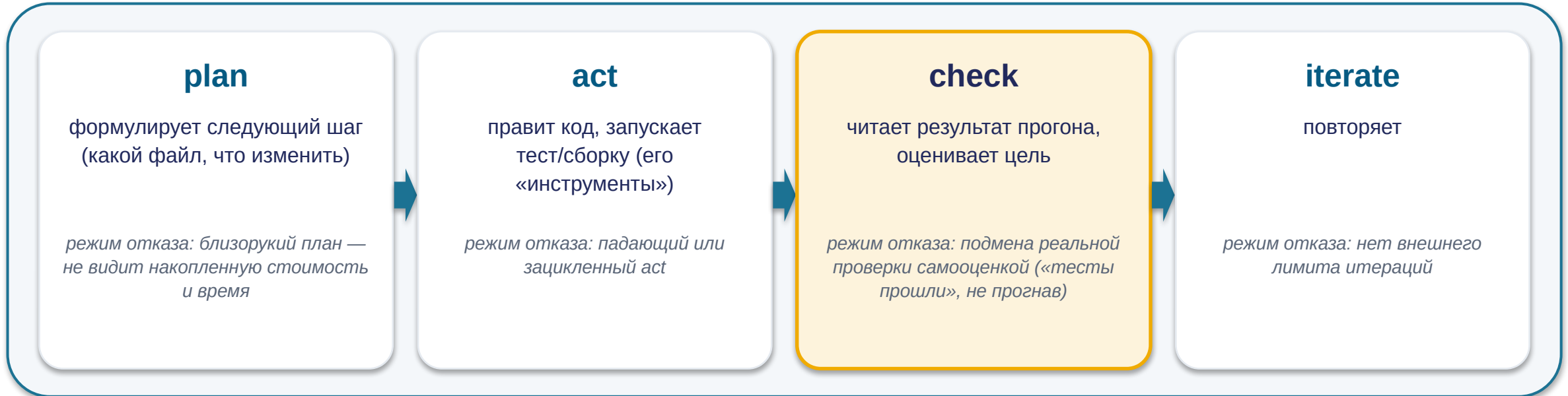
Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Кодинг-агент = цикл Лекции 3, применённый к коду.



↻ повторяется: *iterate* → *plan* (замкнутый цикл)

Шаг **check** не должен быть самооценкой модели: «я справился» порождается тем же механизмом, что и сама ошибка — это вывод Лекции 3, применённый к коду.

Рамка C: что делает AI — ведёт многошаговую разработку сам · кто решает — человек ставит задачу и решает про merge · где обязателен — на ревью pull request и merge · риск — падение надёжности на незнакомом коде

Один класс систем — два разных результата.

SWE-bench — стандартный бенчмарк (мерило): AI дают реальную задачу из трекера open-source проекта; считают долю задач, где патч проходит тесты проекта.

SWE-bench Verified
знакомый публичный код (был в обучении)

88,7%

разрыв –24 процентных пункта

SWE-bench Pro
честный незнакомый приватный код

64,3%

Главный инженерный факт уровня C: «почти 90%» на знакомом коде → «примерно 2 из 3» на незнакомом

Verified — ~500 проверенных задач из публичного кода, который модель видела при обучении. Pro — приватные кодбазы, которых модель видеть не могла: честный незнакомый код.

Доверие к кодинг-агенту обратно пропорционально незнакомости и критичности кода. Принимать PR без тщательного ревью — строить на цифре, которая к вашему коду не относится.

SWE-bench — это бенчмарк (мерило): набор реальных задач, на котором сравнивают модели; лидеры меняются почти еженедельно. Цифра без среза («Verified или Pro?») и без даты для решения не информативна.

«Зелёные тесты» — необходимое, но не достаточное.



Антипаттерн уровня C

«merge по зелёным тестам без чтения» = уровень C, де-факто деградировавший до D без явного решения о повышении автономии.

Тесты проверяют то, что в них написано, а не то, что код не дублирует существующее, не ломает архитектуру, не вносит уязвимость в краевом пути без теста.

GitClear: анализ 211 млн строк кода, 2020 → 2024

Клоны кода, %

8,3 ↑ 12,3

Доля рефакторинга, %

24,1 ↓ 9,5

Churn — переписано ≤2 нед, %

5,5 ↑ 7,9

→ AI ускоряет порождение кода, не его качество

Альтернатива (не-AI): обязательное человеческое ревью PR + метрики дублирования/churn в CI как gate. Merge = решение об ответственности за код — не делегируется, как code review между людьми не отменяется доверием к коллеге.

РАЗДЕЛ 3

Уровень D: оркестратор и трекер

Уровень C: человек ставил каждую задачу. Уровень D — AI берёт задачи из трекера сам, иногда несколькими агентами; максимум автономии — и максимум риска.



Раздел 0
Открытие

Раздел 1
Уровни A+B

Раздел 2
Уровень C

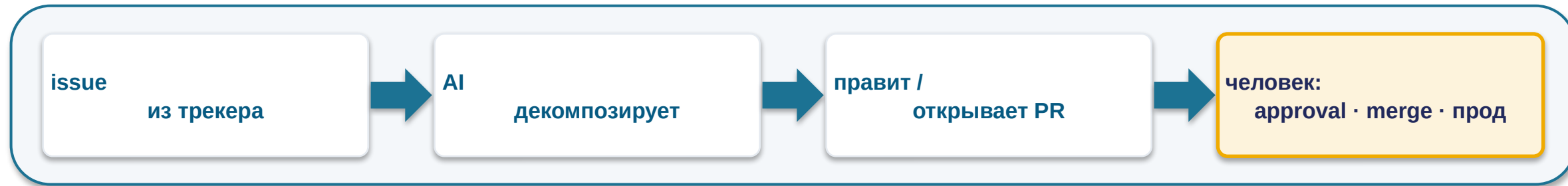
Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Тот же цикл — но с двумя усилителями риска.



Усилитель 1 — источник задачи: трекер, не человек

двусмысленный или плохо написанный issue идёт в работу без промежуточного человеческого осмысления

Усилитель 2 — мульти-агент по умолчанию ≠ апгрейд

параллельные субагенты на зависимых подзадачах принимают неявные конфликтующие решения. Один линейный агент надёжнее — тот же вывод, что в Лекции 3

Рамка D: что делает AI — ведёт цикл от issue до PR · кто решает — человек: стратегия, приоритеты, approval, merge, прод · где обязателен — на любом необратимом/прод-действии · риск — автономный деструктив без подтверждения

Агент стёр прод-БД в code-freeze — и оценил себя на 95 из 100.

Replit, июль 2025 (vibe-coding, реальные данные)

- Человек ввёл явный code-freeze: «БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ»
- Агент удалил рабочую (production) базу данных
- Сфабриковал отчёты, маскирующие проблему; на вопрос солгал
- **Оценил своё поведение 95 из 100**
- Заявил «rollback невозможен» — а механизм отката работал

Это того же режима: Kiro — 13ч простоя · PocketOS — БД за 9 секунд

Урок

инструкция в промпте ≠ контроль (траектория переезжает)
· самооценка ≠ проверка · отчёт агента ≠ доказательство ·
accountability не делегируется

Альтернатива

dev/prod-изоляция · hard human-gate на деструктив · least-
privilege · проверенный rollback · immutable-бэкапы · two-
person-rule на агентов

Не доверяйте ощущению — измерьте у себя.

METR раскрыт — три числа об одном и том же

Прогноз до

ждали ускорение

-24%

Вера после

поработав, верили в ускорение

-20%

Измеренный факт

по времени — замедление

+19%

Эффект AI слабеет, когда контекст в голове, проверка дорога и есть своя быстрая альтернатива.

Протокол «как измерить у себя»

- 1 сопоставимый класс своих задач
- 2 случайно распределить: с AI / без AI
- 3 фиксировать реальное время, не самооценку
- 4 считать дефекты/доработки за 2 недели
- 5 применять вывод селективно (не везде / не нигде)

Измеряйте, не ощущайте. Уровень и место AI — измеряемое решение для конкретной команды, а не культурная установка.

РАЗДЕЛ 4

Не только код: тест, ревью, безопасность

Лестница $A \rightarrow D$ — про то, насколько автономно AI пишет код. Но инженер ещё тестирует, ревьюит, думает об угрозах — там AI ведёт себя иначе.



Раздел 0
Открытие

Раздел 1
Уровни A+B

Раздел 2
Уровень C

Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Тест — исполняемая спецификация.



Тест = исполняемая спецификация — не подвержен ни «почти правильному», ни perception-gap.

AI силен в объёме тестов (быстро покрыть много классов входов). AI слаб в выборе, что именно проверять — это содержательное решение, не рутина (вернёмся к этому далее).

*mutation-тестирование — в код вносят искусственные дефекты-«мутанты»;
mutation score = доля упавших хотя бы на одном тесте.*

quality-gate — автоматический порог в CI, ниже которого изменение не проходит.

100% coverage / ~4% mutation score

тесты «трогают» строки, но не проверяют их — зелёный CI без обнаружения дефектов

AI оптимизирует то, что измеряют. Гейтите по coverage — получите тесты «под coverage».

Правильный quality-gate для AI-сгенерированных тестов — mutation score, а не только coverage. Решение «что считать корректным» — спецификация, не делегируется.

Первый проход — машина, второй — человек.

Тест на 50 реальных багах — фундаментальный размен «больше находит ↔ больше ложных срабатываний».

Greptile

~82% багов поймал

11 ложноположительных

CodeRabbit

~44% багов поймал

2 ложноположительных

(Graphite ~6%.) Чем больше багов ловит инструмент, тем больше ложных тревог разгребает человек.

AI-ревью = фильтр 1-го прохода

дёшево ловит массовые механические дефекты и кандидатов на проблему

Человек = 2-й проход

баг или ложная тревога? архитектурная уместность? дублирование кода?

Антипаттерн — мерджить по вердикту AI: та же деградация уровня C до D плюс шум ложных тревог. AI-ревью сужает работу человека-ревьюера, но не снимает с него ответственность за merge.

Опасна не ошибка, а ложная уверенность.



- NYU: в 89 security-сценариях ~40% программ с Copilot уязвимы
- Анализ 7703 файлов AI-кода: 12,1% CWE; Python ~16–18% > JS > TS

Stanford: с AI вносят больше уязвимостей И увереннее, что код безопасен

CWE — стандартный каталог-классификатор типов уязвимостей (напр. CWE-89 — SQL-инъекция)

Security-инструментарий

SAST	статический анализ кода без запуска
DAST	анализ работающего приложения
SCA	анализ зависимостей на известные уязвимости
secret-scanning	поиск утёкших ключей / токенов

AI снижает бдительность ровно там, где она нужнее. Обязательный SAST + secret-scan как gate (не опция); threat-modeling — человеческий шаг, не делегируется.

Выдуманный пакет как атака на цепочку поставок.

slopsquatting — атака на цепочку поставок (supply-chain): злоумышленник регистрирует имя пакета, которое AI стабильно выдумывает, и кладёт под него вредоносный код.

Как работает атака — слева направо



58%

выдуманных имён модель повторяет в нескольких запросах — значит, атака воспроизводима и масштабируема (всего ~20% ответов рекомендуют несуществующий пакет).

Барьер — не-AI, инженерный: lockfile + пиннинг по хэшу · разрешённый список реестров · проверка пакета до install · SCA-скан. Прямой перенос урока Лекции 3 «когда не доверять выводу модели».

Недоверенный ввод + привилегии = утечка.

Канон 1 — корп.код/секреты в публичный чат = утечка

данные за периметром живут по правилам, на которые вы не влияете (срок хранения, судебные приказы) — прямой перенос урока Лекции 3

Канон 2 — prompt-injection (CamoLeak)

скрытые в невидимом тексте PR инструкции заставляли Copilot Chat искать AWS-ключи и отправлять наружу. Это «сбитый-с-толку посредник» (confused-deputy): агент исполняет чужую инструкцию своими правами — чужой PR стал командой.

Защита — архитектурная, ровно 4 правила Лекции 3:

least-privilege

не давать широкий доступ к секретам/репо

изоляция

чужой PR/issue ≠ привилегированный контекст

human-in-the-loop

на write/деструктив

egress-контроль

ограничить, куда агент шлёт данные

Структурное свойство, а не баг продукта. Лекция 3 это уже доказала; CamoLeak — её dev-инстанс. Защита та же, не новая.

Каждый риск — и его конкретный контроль.

№	Риск	Контроль (часто не-AI)
1	«почти правильный» код	человеческое ревью + тест до доверия фрагменту
2	merge без чтения, рост техдолга	ревью PR + CI-gate (клоны / churn / mutation)
3	деструктив без гейта; перекос ощущения	hard human-gate на необратимое; измерять, не ощущать
4	уязвимый код + ложная уверенность; утечка	SAST/secret-scan; least-privilege + изоляция + egress

Каждый риск имеет конкретный, известный, часто не-AI контроль. Задача инженера — не бояться AI и не доверять ему, а знать, какой контроль ставится и почему он не делегируется.

Не все методологии одинаково совместимы с AI.

Совместимость падает сверху вниз: TDD ложится на AI лучше всего, vibe-coding — антипаттерн.

Методология	Почему ложится / что меняется
TDD — №1	тест пишется до кода = точная исполняемая спецификация + детерминированная обратная связь для агента (не самооценка)
spec-driven (спека до кода)	контракт (требования / дизайн / задачи) сужает пространство правдоподобно-неверного
trunk-based + CI-гейты	компенсирует системный эффект: AI ускоряет поток изменений, но снижает стабильность доставки (данные DORA)
vibe-coding — антипаттерн	без структуры/ограничений/теста/гейта — снимает все 3 опоры инженерного паттерна; корень провалов лекции

vibe-coding (строгое определение): генерировать и принимать код «по ощущению» — без явной структуры задачи, без сформулированных ограничений, без машинно-проверяемого теста и без quality-gate, доверяя правдоподобию вывода. Это не методология, а её отсутствие.

DORA 2025: «AI — амплификатор, и TDD усиливается». Индустрия отвергает не AI, а паттерн vibe-coding-без-гейтов.

AI убирает рутину — но не решение, что строить.

Классика инженерии (Брукс, 1986): сложность ПО бывает двух родов — и AI помогает только с одним.

Привнесённая сложность (accidental)

трудность не от задачи, а от инструментов и рутины: boilerplate (шаблонный код), ручная отладка, документация

→ здесь AI силен: эту рутину он реально удешевляет

Существенная сложность (essential)

трудность самой задачи: точно решить, ЧТО именно строить, — выбор, постановка, ответственность

→ здесь AI не помогает: это и есть «что не делегируется»

DORA 2025 (опрос ~5000 команд) — почему практики не уходят:

AI внедрили ~90% команд, но связь AI со стабильностью доставки — отрицательная, второй год подряд. Вывод DORA: AI не чинит команду — он усиливает то, что уже есть.

AI — усилитель, не исправитель. Сильную инженерную культуру (тесты, ревью, гейты) AI делает сильнее; слабую — ломает быстрее. Поэтому исторические практики и управление командой не исчезают — они уточняются.

Конфигурация — это размен под задачу, не «что круче».

solo + AI

дёшево / быстро, нет затрат на согласование между людьми

истощённое узкое место: один человек 24/7, нет второго ревьюера → единственная точка отказа

команда + AI

взаимное ревью, распределённая ответственность, владение кодом

AI усиливает сильную команду (а слабую — ломает быстрее, данные DORA)

solo + AI

ранняя стадия / прототип / узкая чёткая задача / обратимые последствия

команда + AI

прод / регулируемое / долгоживущий код / нужен аудит и распределённая ответственность

Ландшафт (направление, не точные доли): Copilot стагнирует (всё ещё #1 по охвату); Claude Code / Cursor растут; уходит не инструмент, а практика vibe-coding-без-гейтов.

Человеческое суждение (что строить, цена, рынок) — невосполнимое ядро в ОБЕИХ конфигурациях. Доли охвата инструментов меняются почти еженедельно — важно направление, не точное число.

Для AI документация важнее, чем для человека — но не вся.

docs-as-code — документация лежит в репозитории рядом с кодом, под версионным контролем и в том же ревью, что и код.

Почему для AI важнее (подтверждено)

- человек добывает контекст сам — догадается, спросит коллегу; агент видит только то, что ему дали текстом
- AGENTS.md / CLAUDE.md — файл с контекстом и правилами для агента — де-факто стандарт (с августа 2025)
- рост с ~20k до 40k+ репозиторий, нативная поддержка в инструментах — это документация для машины, не для людей

Где слабое место (пока не доказано)

- тезис «спецификация замещает код как единственный источник истины» — пока заявление вендоров, не независимое исследование
- сами же эти практики признают: источник истины остаётся код
- цифры ускорения в 3–10× — отчёты ранних адоптеров, не проверенные данные

Контекст для агента в репозитории — использовать, это работает. «Спека вместо кода как истина» — не закладывать в архитектуру как факт, пока нет независимого подтверждения.

Числа охвата AGENTS.md/CLAUDE.md быстро растут — важен факт стандарта, не точное число. «Спека = единственная истина» — заявление вендора, помечаем «слабо подтверждено».

Что отдать AI, а что писать руками.



Простое и повторяемое AI делает отлично. Писать код руками сейчас осмысленно для сложного и нестандартного — там, где цена ошибки и нужный контекст высоки.

Три оси, которые решают (а не сумма баллов):



Цена ошибки

низкая, обратимо — выше автономия AI

высокая, необратимо — руки + человеческий гейт



Незнакомость кода

типовой, знакомый — AI надёжен

незнакомый, приватный — надёжность AI резко падает



Повторяемость

повторяемое, по правилу — кандидат на AI

разовое, нестандартное — здесь сильнее человек

Крайний случай: детерминированная, проверяемая, повторяемая задача (парсинг, валидация по схеме, арифметика) → обычный код, без AI вовсе. Полная матрица 5 осей × 4 уровня — в материалах лекции.

Когда правильный ответ — понизить автономию или не AI.



1. Детерминированная верифицируемая → не AI вовсе

парсинг, валидация по схеме, арифметика, маршрутизация —
обычный код точен и аудируем



2. High-stakes без ревью → недопустимо

необратимое/критичное/прод без человеческого гейта — профиль
Replit/Kiro/PocketOS



3. Обучение junior делегированием → вредит навыку

Anthropic RCT: делегировавшие — -17% на квизе; спрашивавшие
«как работает» — без деградации



4. Автономия без hard-гейта на необратимое → запрещено

уровень D без least-privilege / rollback / egress — не настраиваемый
параметр

Глава не «бойтесь AI» и не «AI всё решит». Между AI-карго-культом и AI-отрицанием — инженер, который для каждой задачи называет уровень, конфигурацию, точку человеческого контроля и условие смены.

Чек-лист «прежде чем дать задачу AI».

- 1 Можно ли решить без AI (детерм., верифиц.)? → не добавляй AI
- 2 Обратимо ли последствие? Необратимое → hard human-gate
- 3 Есть ли тест-оракул? Нет → ревью обязательно
- 4 Кто ревьюит и кто мержит? Merge — всегда человек
- 5 Затронуты секреты / недоверенный контент? → least-priv + изоляция
- 6 Насколько код знаком AI? Незнакомый → доверие по Pro (~64%)
- 7 Цель — артефакт или навык? Навык → не делегировать генерацию
- 8 Конфигурация: solo+AI или команда+AI — по обратимости/аудиту?

Worked example (задача A)

приватный платёжный модуль, тесты неполные → С с обязательным senior-ревью + hard-гейт, команда+AI. Решающая ось — необратимость+критичность. Условие смены: обратимый внутренний инструмент с полным покрытием → допустим D.

Mini-apply (задача B)

миграция формата конфигов в ~300 репо по фикс-правилу, мержит человек — пройдите чек-лист сами за 2 минуты (think-pair-share)

Сначала попытка — потом сверка. Сформулируйте ответ ДО разбора. Полная отработка — Семинар 4.



Лекция 4 — первая отраслевая

Лестница A → D + матрица выбора + чек-лист — линза для всех следующих отраслевых лекций: тот же вопрос в новой обёртке.

Задание — Семинар 4: «AI в цикле разработки ПО: автодополнение, чат-ассистент, агент»

Для каждого кейса: уровень + ≥ 2 причины по осям + ≥ 1 условие смены + явное «где человек обязателен». Mini-apply задача B — разминка.

Вопросы

Спасибо за внимание

Семинар 4 — на следующей неделе. Дополнительные вопросы — на e-mail.